4. Driver protocol and instructions

4.1. Description of the communication protocol type

The motor communication is the CAN 2.0 communication interface, the baud rate is 1Mbps, and the extended frame format is adopted as follows:

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	Communication type	data area 2	Destination address	data area 1

The control modes supported by the motor include:

- Operation control mode: set 5 parameters of motor operation control;
- Current mode: the specified Iq current of the given motor;
- Velocity mode: the specified running speed of the given motor;
- Position mode: Given the specified position of the motor, the motor will run to the specified position;

4.1.1. Communication type 0: Get device ID

Gets the device's ID and 64-bit MCU unique identifier

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x0	bit15~8: identifies host CAN_ID	target motor CAN_ID	0

Reply frame:

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x0	target motor CAN_ID	0XFE	64-bit MCU unique identifier

4.1.2. Communication Type 1: operation control mode motor control instruction

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x1	Byte2: Torque (0~65535) corresponds to (-17Nm~17Nm)	target motor CAN _ID	Byte0~1: target Angle [0~65535] corresponds to (-4π~4π) Byte2~3: Target angular velocity [0~65535] corresponds to (-44rad/s~44rad/s) Byte4~5: Kp [0~65535] corresponds to (0.0~500.0) Byte6~7: Kd [0 to 65535] corresponds to the above data (0.0 to 5.0). After the conversion, the high byte is in front and the low byte is in

Response frame: Response motor feedback frame (see communication type 2)

Communication Type 2: motor feedback data

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x2	Bit8~Bit15: CAN ID of the current motor bit21~16: fault information (0 none 1 has) bit21: uncalibrated bit20: Uncalibrated bit20: Gridlock overload fault bit19: magnetic coding fault bit18: overtemperature bit17: Three-phase overcurrent fault bit16: undervoltage fault bit22~23:	host CAN _ID	Byte0~1: The current Angle [0~65535] Corresponding to (-4π~4π) Byte2~3: Current angular velocity [0~65535] corresponds to (-44rad/s~44rad/s) Byte4~5: Current torque [0~65535] corresponds to (-17Nm~17Nm) Byte6~7: Current temperature: Temp(Celsius) *10 If the value is higher than 10, the high byte is first and the low byte is last

		Mode status 0: Reset mode [reset] 1: Cali mode [calibration] 2: Motor mode [Run]		

4.1.3. Communication Type 3: Motor enabled to run

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x3	bit15~8: identifies the main CAN_ID	and target motor CAN_ID	

Response frame: Response motor feedback frame (see communication type 2)

4.1.4. Communication Type 4: Motor stops running

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x4	bit15~8: used to identify the main CAN_ID	target motor CAN_ID	When the motor is running normally, 0 must be cleared in the data field. Byte[0]=1: The fault is cleared.

Response frame: Response motor feedback frame (see communication type 2)

Version number read frame

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x4	bit15~8: used to identify the main CAN_ID	target motor CAN_ID	Byte[0]=0x00 Byte[1]=0xC4

Response frame:

data field	29 位 ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x2	Bit8~Bit15: CAN ID of the	target	Byte0=0x00

		current motor bit21~16: fault information (0 none 1 has) bit21: uncalibrated bit20: Uncalibrated bit20: Gridlock overload fault bit19: magnetic coding fault bit18:overtemperature bit17: Three-phase overcurrent faultbit16: undervoltage fault bit22~23: Mode status 0: Reset mode [reset] 1: Cali mode [calibration] 2: Motor mode [Run]	motor CAN_ID	Byte1=0xC4 Byte2=0x56 Byte3~6:Motor version number Order from high to low

4.1.5. Communication type 6: Set motor mechanical zero

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x6	bit15~8: Identifies the main CAN_ID	and target motor CAN_ID	Byte[0]=1

Response frame: Response motor feedback frame (see communication type 2)

Communication type 7: Set motor CAN_ID

Change the current motor CAN_ID, effective immediately.

data field	29-bit ID			8Byte data field
Size	Bit28~bit 24	bit23~8	bit7~0	Byte0~Byte7
descripti on	0x7	bit15~8: used to identify main CAN_ID Bit16~23: preset CAN_ID	Target motor CAN_ID	

Answer frame: Answer motor broadcast frame (see communication type 0)

4.1.6. Communication type 17: Single parameter read

Data field	29 bit ID	8Byte data field

Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x11	bit15~8: Used to identify the main CAN_ID	target motor CAN_ID	Byte0~1: index. For details, see the readability parameter table below Byte2~3:00 Byte4~7: In data above 00, the low byte is first and the high byte is second (

Reply frame:

Data field	29 bit ID			8Byte data field
Size	Bit28~bit 24	bit23~8	bit7~0	Byte0~Byt e7
descripti on	0x11	bit15~8: indicates that the master CAN_ID Bit23~16:00 indicates that the master CAN_ID is successfully read. 01 indicates that the master can_ID	Byte0~1: Byte2~3:00 Byte4~7:Parameter data. 1 byte of data above Byte4 is preceded by low bytes and followed by high bytes at	

4.1.7. Communication type 18: Single parameter write (lost in power failure)

With type 22, the parameter starting with function code 0x20 of the parameter table in the upper computer module can be saved

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x12	bit15~8: Used to identify the main CAN_ID	target motor CAN_ID	Byte0~1: index. For details, see the readability parameter table below Byte2~3: 00 Byte4~7: Parameter data In the preceding data, the low byte is in the front and the high byte is in the rear

Response frame: Response motor feedback frame (see communication type 2)

4.1.8. Communication type 21: Fault feedback frame

data field	29-bit ID	8Byte data field

Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x15	bit15~8: motor CAN_ID	identifies the main CAN_ID	Byte0~3: fault value (non-0: faulty; 0: faulty). Normal) Bit 16: A-phase current sampling overcurrent Bit 14: Motor stall overload algorithm protection Bit 9: Position initialization fault Bit 8: Hardware identification fault Bit 7: Encoder uncalibrated: Motor encoder not calibrated Bit 5: C-phase current sampling overcurrent Bit 4: B-phase current sampling overcurrent bit3: overvoltage fault bit2: undervoltage fault bit1: driver chip fault bit0: motor overtemperature fault, Default 103 ° C Byte4~7: warning Value bit0: motor overtemperature warning, the default is 93 ° c

4.1.9. Communication type 22: Motor data save frame

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x16	bit15~8: identifies the main CAN_ID	and target motor CAN_ID	01 02 03 04 05 06 07 08

Response frame: Response motor feedback frame (see communication type 2)

4.1.10. Communication type 23: Motor baud rate modification frame (re-power-on effect)

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x17	bit15~8: used to identify the main	target motor CAN_ID	01 02 03 04 05 06 F_CMD Among them, the F_CMD byte is the motor baud rate Among them, 01 is 1M 02 is 500K 03 is

		CAN_ID		250K	04 is 125K

Response frame: Response motor feedback frame (see communication type 0)

4.1.11. Communication type 24: The motor actively reports frames

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x18	bit15~8: identifies the main CAN_ID	target motor CAN_ID	01 02 03 04 05 06 F_CMD Among them, the F_CMD byte is the motor reporting switch 00 is to disable active reporting (default) 01 To enable active reporting, the default reporting interval is 10ms

Response frame:

data field	29 位 ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x18	Bit8~Bit15: CAN ID of the current motor bit21~16: fault information (0 none 1 has) bit21: uncalibrated bit20: Uncalibrated bit20: Gridlock overload fault bit19: magnetic coding fault bit18:overtemperature bit17: Three-phase overcurrent faultbit16: undervoltage fault bit22~23: Mode status 0: Reset mode [reset] 1: Cali mode [calibration] 2: Motor mode [Run]	target motor CAN_ID	Byte0~1: The current Angle [0~65535] Corresponding to (-4π~4π) Byte2~3: Current angular velocity [0~65535] corresponds to (-44rad/s~44rad/s) Byte4~5: Current torque [0~65535] corresponds to (-17Nm~17Nm) Byte6~7: Current temperature: Temp(Celsius) *10 If the value is higher than 10, the high byte is first and the low byte is last

4.1.12. Communication type 25: Motor protocol modification frame (re-power-on effect)

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x19	bit15~8: used to identify the main CAN_ID	target motor CAN_ID	01 02 03 04 05 06 F_CMD Among them, the F_CMD byte is the motor protocol type Among them, 0 is a private protocol (default) 1 is the Canopen protocol 2 is the MIT protocol

Response frame: Response motor feedback frame (see communication type 0)

4.1.13. Read and write a single parameter list

index		Description	Type	Number of bytes		R/W Read and write permission
0X7005	run_mode	0: operation mode 1: position mode (PP) 2: Velocity mode 3: Operation mode Current mode 5: Position mode (CSP)	uint8	1		W/R
0X7006	iq_ref	Current mode Iq command	float	4	-23 to 23A	W/R
0X700A	spd_ref	Rotational Velocity mode Rotational speed command	float	4	-44to 44rad/s	W/R
0X700B	limit_torque	torque limit	float	4	0 to 17Nm	W/R
0X7010	cur_kp	Kp	float	4	The default value is 0.17	W/R
0X7011	cur_ki	Ki	float	4	The default value is 0.012	W/R
0X7014	cur_filt_gain	filt_gain	float	4	0 to 1.0, The default	W/R

index		Description	Type	Number of bytes		R/W Read and write permission
					value is 0.1	
0X7016	loc_ref	Position Mode Angle instruction	float	4	rad	W/R
0X7017	limit_spd	Location mode (CSP) speed limit	float	4	0 to 44rad/s	W/R
0X7018	limit_cur	Velocity position mode Current limitation	float	4	0 to 23A	W/R
0x7019	mechPos	Mechanical Angle of the loading coil	float	4	rad	R
0x701A	iqf	iq Filter	float	4	-16 to 16A	R
0x701B	mechVel	Speed of the load	float	4	-44 to 44rad/s	R
0x701C	VBUS	Bus voltage	float	4	V	R
0x701E	loc_kp kp	at	float	4	The default value is 40	W/R
0x701F	spd_kp	Indicates the speed kp	float	4	The default value is 6	W/R
0x7020	spd_ki	ki	float	4	The default value is 0.02	W/R
0x7021	spd_filt_gain	Speed filter value	float	4	The default value is 0.1	W/R
0x7022	acc_rad	velocity mode acceleration	float	4	The default value is 20rad/s^2	W/R
0x7024	vel_max	Location mode (PP) speed	float	4	The default value is 10rad/s	W/R
0x7025	acc_set	Location mode (PP) acceleration	float	4	The default value is 10rad/s^2	W/R
0x7026	EPScan_time	Indicates the report time. 1 indicates 10ms. Plus 1 increments by 5ms	uint16	2	The default value is 1	W/R
0x7028	canTimeout	can The timeout threshold, 20000 is 1s	uint32	4	The default value is 0	W/R

index		Description	Type	Number of bytes		R/W Read and write permission
0x702C	alveolous_open	Cogging Compensation Switch	uint8	1	The default is 0	W/R
0x702D	iq_test	Motor Initialization Calibration Switch	uint8	1	The default is 0	W/R
0x702E	dcc_set	Deceleration in Motor PP Mode (Normally Disabled)	float	4	The default is 10rad/s^2	W/R

4.1.14. Read example:

Take reading loc_kp as an example:

Read instruction is

Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
	0x11	0x00FD	0x7F	1E 70 00 00 00 00 00 00
Description	Type 17	Host id 0xFD	Target motor CAN_ID 7F	Byte0~1: index, corresponding to loc_kp

The feedback instruction is

Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
	0x11	0x007F	0xFD	1E 70 00 00 00 00 F0 41
Description	Type 17	bit15~8: Target motor CAN_ID 7F	Host id 0xFD	Byte0~1: index, corresponding to loc_kp Byte4~7:loc_kp value 30, high right byte, (32-bit single precision) hexadecimal IEEE-754 standard floating point number

4.2. Motor Function Description

(If the following features are unavailable, please upgrade to the latest version via the official Git repository.)

4.2.1. Actively report

The motor automatically reports off by default, and reports on type 24

The reporting type is Type 2. The default reporting interval is 10ms. You can change the reporting period by using EPScan_time of type 18

4.2.2. Type 2 Change Description

Type 2 is changed to a periodic cycle -4π-4π, by which the number of turns can be counted

Note that the location interface needs to be changed

P_MIN is -12.57f

P_MAX is 12.57f

4.2.3. Protocol Switching *(Requires CAN adapter)*

- Methods:
  - Modify `protocol_1` via host computer.
  1. Send **Type 25** command.
- **Reboot required** after switching.
- **Post-switch CAN commands**:
  - **CANopen**: Send extended frame (protocol switch frame).
  - **MIT Protocol**: Send standard frame (**Command 8**).

4.2.4. Zero Calibration Rules

- **Zero calibration is supported in CSP and Motion Control modes, but**

blocked in PP mode.

- **In old firmware versions, zero calibration would cause a large deviation between the motor's target and actual position, triggering an immediate movement of the motor to the target position.**
- **In new firmware versions, when zero calibration is performed in CSP or Motion Control mode, the motor's target position is instantly updated to 0, so the motor remains stationary. Zero calibration is unavailable in PP mode.**

4.2.5. The CANopen ID of the motor is consistent with the CAN ID.

3. **The CANopen ID is kept the same as the CAN ID under the private protocol.**

4.2.6. Motor Initialization Calibration

- When the motor sets iq_test to 1, the initialization time is extended, resulting in a more accurate motor initial reference.

4.2.7. Cogging Calibration Function

2. Before calibration, set iq_test to 1 and restart the motor; the motor must be under no-load conditions. Then perform cogging calibration via the host computer. After successful calibration, max_alve will be non-zero. Next, set alveolous_open to 1 to activate the function in Motion Control mode.

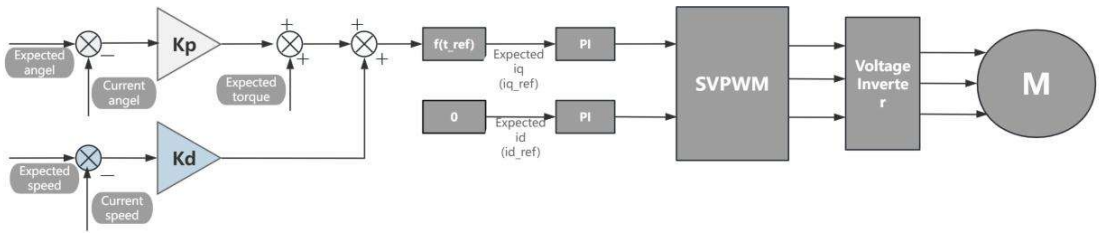
4.2.8. PP Mode Deceleration

- By setting acc_stutus to 1, the acceleration and deceleration for PP mode can be configured separately. The deceleration value is stored at 0x702E. When acc_stutus is set to 0, the acceleration parameter controls both

acceleration and deceleration.

4.3. Control mode instructions

4.3.1. Operation control mode

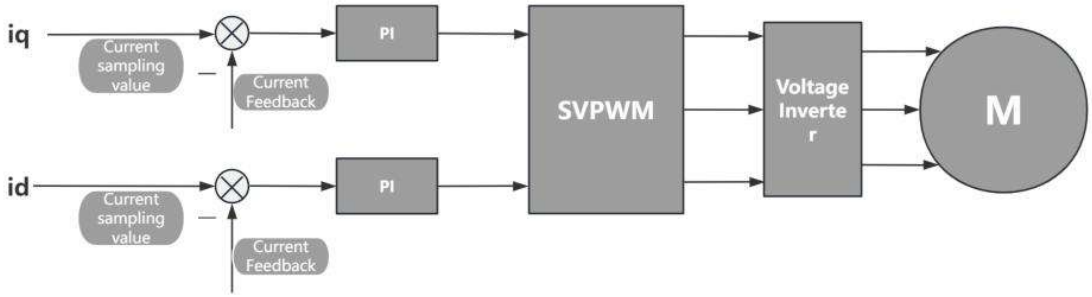


The motor is in operation control mode by default after power-on.

Send motor Enable Run frame (communication type 3) --> Send operation mode motor control command (communication type 1) --> Receive motor feedback frame (communication type 2)

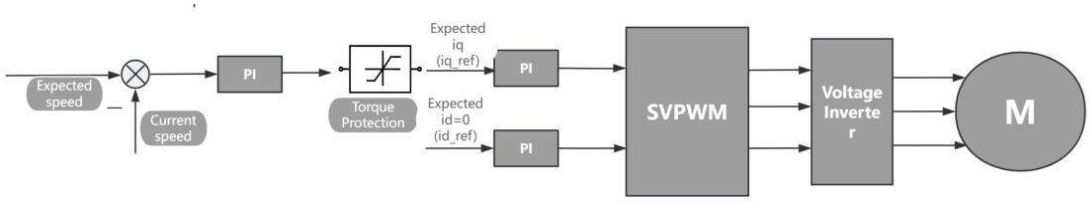
Operation control mode description: The control logic of the operation and control mode is t_ref=Kd * (v_vset-v_actual)+Kp * (p_set-p_actual)+t_ff. Tref is converted to the expected iq current through an internal formula and output through the current loop Simple control demonstration: Set t_ff to 0, v_vset to 1, Kd to 1, p_set to 0, Kp to 0. If there is no external load on the motor, it will run at a speed of 1rad/s. If there is an external load, kd needs to be increased to resist the external load Set t_ff to 0, v_vset to 0, Kd to 1, p_set to 0, Kp to 0, the motor is in damping mode. When the motor is externally rotated, a damping is applied, which increases with the increase of kd. It should be noted that the motor generates electricity under this condition and requires power supply to prevent overvoltage Set t_ff to 0, v_vset to 0, Kd to 1, p_set to 5, Kp to 1. If there is no external load on the motor, it will run to the target position of 5. Increasing kp will increase the force required to maintain the target position, and kd is damping. Without kd, the motor will sway to the target position

4.3.2. Current mode



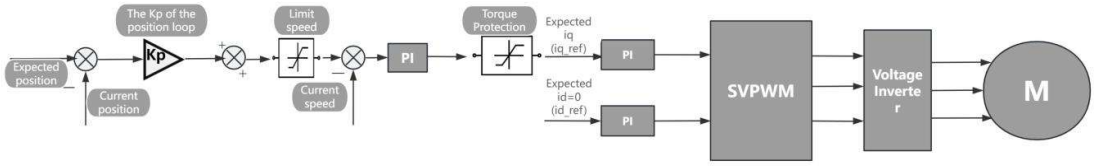
Send motor mode parameter write command (communication type 18) Set the runmode parameter to 3 --> Send motor Enable run frame (communication type 3) --> Send motor mode parameter write command (communication type 18) set the iq_ref parameter to the default current instruction

4.3.3. Velocity mode



Send motor mode parameter write command (communication type 18) Set the runmode parameter to 2 --> Send motor Enable run frame (communication type 3) --> Send motor mode parameter write command (communication type 18) set limit_cur parameter as default maximum current instruction --> Send motor mode parameter write command (communication type 18) Set acc_rad parameter as default acceleration instruction --> Send motor mode parameter write command (communication type 18) Set spd_ref parameter as default speed instruction

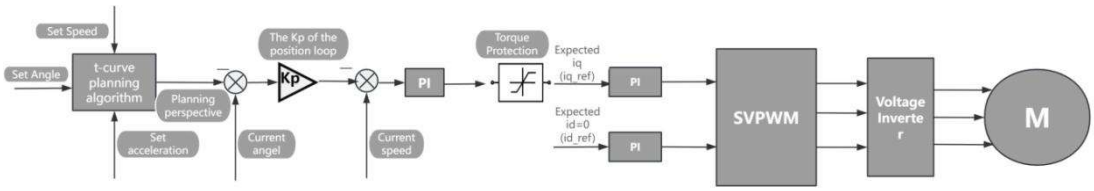
4.3.4. Location Mode (CSP)



Send motor mode parameter write command (communication type 18) Set the runmode parameter to 5 --> Send motor Enable run frame (communication

type 3) --> Send motor mode parameter write command (communication type 18) set limit_spd parameter as default maximum speed instruction --> Send motor mode parameter write command (communication type 18) Sets loc_ref parameter as default position instruction

4.3.5. Location Mode (PP)



Send motor mode parameter write command (communication type 18) Set the runmode parameter to 1 --> Send motor Enable run frame (communication type 3) --> Send motor mode parameter write command (communication type 18) set The vel_max parameter is the default maximum speed instruction --> Send motor mode parameter write command (communication type 18) Set the acc_set parameter to the default acceleration instruction --> Send motor mode parameter write command (communication type 18) Set the loc_ref parameter to the default position instruction

Note: This mode does not support changing the speed and acceleration during operation. If you want to make an emergency stop, you can change vel_max to 0 during the process, and it will stop at the current speed and acceleration plan

4.3.6. Stop running

Sending motor stop frame (communication type 4)

4.4. Program sample

Examples of various mode control motors are provided below (take gd32f303 as an example)

The following are library, function, and macro definitions for the various instances

```
Plain Text
```

```
#define P_MIN -12.57f

#define P_MAX 12.57f

#define V_MIN -44.0f

#define V_MAX 44.0f

#define KP_MIN 0.0f

#define KP_MAX 500.0f

#define KD_MIN 0.0f

#define KD_MAX 5.0f

#define T_MIN -17.0f

#define T_MAX 17.0f

struct exCanIdInfo{

    uint32_t id:8;

    uint32_t data:16;

    uint32_t mode:5;

    uint32_t res:3;

};

can_receive_message_struct rxMsg;

can_trasnmit_message_struct txMsg={
```

```
    .tx_sfid = 0,

    .tx_efid = 0xff,

    .tx_ft = CAN_FT_DATA,

    .tx_ff = CAN_FF_EXTENDED,

    .tx_dlen = 8,

};

#define txCanIdEx (*((struct exCanIdInfo*)&(txMsg.tx_efid)))

#define rxCanIdEx (*((struct exCanIdInfo*)&(rxMsg.rx_efid))) //Parses the
extended frame id into a custom data structure

int float_to_uint(float x, float x_min, float x_max, int bits){

    float span = x_max - x_min;

    float offset = x_min;

    if(x > x_max) x=x_max;

    else if(x < x_min) x= x_min;

    return (int) ((x-offset)*((float)((1<<bits)-1))/span);

}

#define can_txd() can_message_transmit(CAN0, &txMsg)

#define can_rxd() can_message_receive(CAN0, CAN_FIFO1, &rxMsg)
```

The following lists the common types of communication sent:

4.4.1. Motor Enabled Run frame (communication type 3)

```
Plain Text
void motor_enable(uint8_t id, uint16_t master_id)
{
   txCanIdEx.mode = 3;
   txCanIdEx.id = id;
   txCanIdEx.res = 0;
   txCanIdEx.data = master_id;
   txMsg.tx_dlen = 8;
   txCanIdEx.data = 0;
   can_txd();
}
```

4.4.2. Operation control mode Motor control instruction (communication type 1)

```
Plain Text
void motor_controlmode(uint8_t id, float torque, float MechPosition, float
speed, float kp, float kd)

{

   txCanIdEx.mode = 1;

   txCanIdEx.id = id;

   txCanIdEx.res = 0;

   txCanIdEx.data = float_to_uint(torque,T_MIN,T_MAX,16);

   txMsg.tx_dlen = 8;
```

```
    txMsg.tx_data[0]=float_to_uint(MechPosition,P_MIN,P_MAX,16)>>8;

    txMsg.tx_data[1]=float_to_uint(MechPosition,P_MIN,P_MAX,16);

    txMsg.tx_data[2]=float_to_uint(speed,V_MIN,V_MAX,16)>>8;

    txMsg.tx_data[3]=float_to_uint(speed,V_MIN,V_MAX,16);

    txMsg.tx_data[4]=float_to_uint(kp,KP_MIN,KP_MAX,16)>>8;

    txMsg.tx_data[5]=float_to_uint(kp,KP_MIN,KP_MAX,16);

    txMsg.tx_data[6]=float_to_uint(kd,KD_MIN,KD_MAX,16)>>8;

    txMsg.tx_data[7]=float_to_uint(kd,KD_MIN,KD_MAX,16);

    can_txd();

}
```

4.4.3. Motor stop frame (communication type 4)

Plain Text

```
void motor_reset(uint8_t id, uint16_t master_id)

{

    txCanIdEx.mode = 4;

    txCanIdEx.id = id;

    txCanIdEx.res = 0;

    txCanIdEx.data = master_id;
```

```
    txMsg.tx_dlen = 8;

    for(uint8_t i=0;i<8;i++)

    {

            txMsg.tx_data[i]=0;

    }

    can_txd();

}
```

4.4.4. Motor mode parameter write command (communication type 18, running mode switch)

Plain Text

```
uint8_t runmode;

uint16_t index;

void motor_modechange(uint8_t id, uint16_t master_id)

{

    txCanIdEx.mode = 0x12;

    txCanIdEx.id = id;

    txCanIdEx.res = 0;

    txCanIdEx.data = master_id;

    txMsg.tx_dlen = 8;
```

```
    for(uint8_t i=0;i<8;i++)
    {
        txMsg.tx_data[i]=0;
    }
    memcpy(&txMsg.tx_data[0],&index,2);
    memcpy(&txMsg.tx_data[4],&runmode, 1);
    can_txd();
}
```

4.4.5. Motor mode parameter write command (communication type 18, control parameter write)

Plain Text

```
uint16_t index;
float ref;
void motor_write(uint8_t id, uint16_t master_id)
{
    txCanIdEx.mode = 0x12;
    txCanIdEx.id = id;
    txCanIdEx.res = 0;
```

```
    txCanIdEx.data = master_id;
    txMsg.tx_dlen = 8;
    for(uint8_t i=0;i<8;i++)
    {
          txMsg.tx_data[i]=0;
    }
    memcpy(&txMsg.tx_data[0],&index,2);
    memcpy(&txMsg.tx_data[4],&ref,4);
    can_txd();
}
```

5. Explanation of Canopen Communication Protocol Types

5.1. Introduction to CANopen Communication

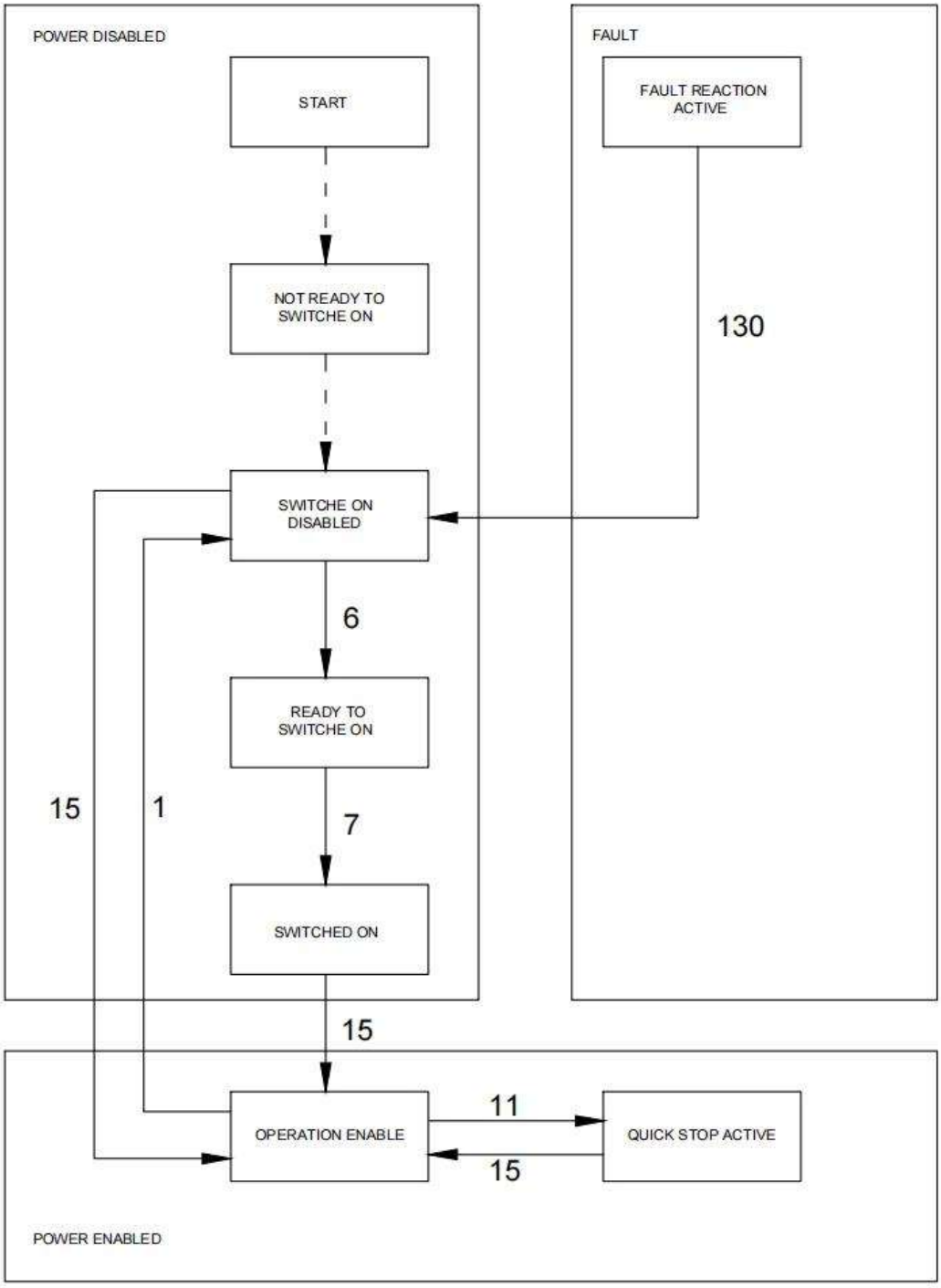
CANopen is a "higher-level protocol" based on the CAN bus. This means that the CAN bus (ISO 11898) acts like a container truck, serving as the "transportation vehicle" for CANopen information. CAN simply implements the transmission of frames with an 11-bit CAN ID, a Remote Transmission (RTR) bit, and 64 data bits (related to higher-level protocols). The CAN bus plays the same role in CANopen as it does in the J1939 protocol. CANopen implements Layer 7 of the OSI model and is compatible with other data link layer protocols besides CAN.Please download the necessary EDS files for CANopen from the official website download center at www.robstride.com.

5.2. Canopen Protocol Message Classification

Classification	Function
NMT Network Management Messages	Manage the network and switch node states. NMT network management messages are typically sent by the master station.
SDO Service Data Object Message	Used for setting device parameters or transmitting critical data. Typically, the master station initiates an SDO message, and the slave station responds. However, a slave station can also initiate an SDO message, with the master station responding, for example, in the transmission of critical data.
PDO (Process Data Object) messages	Transmit process data from various devices, such as temperature and speed. Both the master and slave stations send these messages.
EMCY Emergency Message	Transmits fault information about the transmission equipment. Both the master and slave stations send this message.
SYNC Synchronization Message	Synchronization data, used to synchronize TPDO data on the slave station. Generally sent by the master station.

Classification	Function
NODE GUARDING Message	The master station requests the slave station's status; the master station queries, and the slave station responds.
Heartbeat Message	A device actively sends a heartbeat to indicate that it is online. Both the master and slave devices can send heartbeats.

5.3. State Machine Description



Motor Enable:

When initially powered on, the motor defaults to the **SWITCH_ON_DISABLED** state. To transition to **OPERATION_ENABLE**, modify the **Controlword (6040H)** to **6, 7, or 15** (step-by-step transition), or directly set it to **15** for immediate enablement.

Stopping the Motor:

If the motor is in **OPERATION_ENABLE** state and needs to stop normally, modify the **Controlword (6040H)** to **1**. The motor will return to the disabled state (**SWITCH_ON_DISABLED**).

Emergency Stop (Use with Caution—Risk of Voltage Surge):

During operation, an emergency stop can be triggered by setting the **Controlword (6040H)** to **11**.

Fault Clearance:

If the motor enters a **FAULT** state due to protection mechanisms, modifying the **Controlword (6040H)** can clear standard errors.

Motor Watchdog

Set the CAN communication watchdog via index 1 of object 0x6099. After configuration, a value of 20000 represents 1 second. If no communication frame is received within the set time, the motor will be disabled. The default value is 0 (disabled), and a non-zero value enables the watchdog.

Important Note:

Mode changes for this motor must be performed in the **disabled state (SWITCH_ON_DISABLED)**. Ensure the desired mode is configured **before** enabling **OPERATION_ENABLE** to avoid unexpected behavior.

5.4. Status Feedback Parameters

Index	Name	Attribute	Type	Unit
603F	**Error_code**	Read-only	UINTEGER16	/
6041	**Statusword**	Read-only	UINTEGER16	/
6061	**Modes_of_operation_display**	Read-only	INTEGER8	/
6062	**Position_demand_value**	Read-only	INTEGER32	Pulses (1 rev = 16,384 pulses)
6064	**Position_actual_value**	Read-only	INTEGER32	Pulses (1 rev = 16,384 pulses)
606B	**Velocity_demand_value**	Read-only	INTEGER32	0.1 rpm

Index	Name	Attribute	Type	Unit
606C	**Velocity_actual_value**	Read-only	INTEGER32	0.1 rpm
6077	**Torque_actual_value**	Read-only	INTEGER16	0.1% load ratio (1000 = 6 N·m)
6078	**Current_actual_value**	Read-only	INTEGER16	mA
6079	**DC_link_circuit_voltage**	Read-only	INTEGER32	mV

5.5. Homing Mode (Zero Position Setting)

Index	Name	Attribute	Type	Unit
6040	**Controlword**	Read-write	UINTEGER16	/
6060	**Modes of operation**	Read-write	INTEGER8	/

Homing method:

- Set **Modes of operation** to **6** while the motor is in the **disabled state (SWITCH_ON_DISABLED)**. The motor will then define the current position as the zero point.
- To **hold the zero position**, modify the **Controlword** to **15**, and the motor will maintain its position at the home location.

5.6. Position Mode (PP - Profile Position)

Index	Name	Attribute	Type	Unit
6040	**Controlword**	Read-write	UINTEGER16	/
6060	**Modes of operation**	Read-write	INTEGER8	/
6067	**Position_window**	Read-write	UINTEGER32	Pulses (1 rev = 16,384 pulses)
6068	**Position_window_time**	Read-write	UINTEGER16	ms
6071	**Target_torque**	Read-write	INTEGER16	0.1% load ratio (1000 = 6 N·m)
607A	**Target_position**	Read-write	INTEGER32	Pulses (1 rev = 16,384 pulses)
6081	**Profile_velocity**	Read-write	UINTEGER32	0.1 rpm

Index	Name	Attribute	Type	Unit
6083	**Profile_acceleration**	Read-write	UINTEGER32	0.1 rpm/s

Steps to Configure Position Mode (PP):

2. While the motor is in the **disabled state (SWITCH_ON_DISABLED)**, set **Modes of operation** to **1**.
    - **Mandatory parameters**:
        - **Target_torque** (absolute max torque in position mode)
        - **Profile_velocity** (absolute speed in position mode)
        - **Profile_acceleration** (absolute acceleration in position mode)
    - **Optional parameters**:
        - **Position_window** (if not set, window check is disabled)
        - **Position_window_time** (if not set, window check is disabled)
2. Set **Controlword (6040)** to **15** to enable operation.
4. **Set Target_position** (absolute position) to move the motor to the desired position.

5.7. Position Mode (CSP - Cyclic Synchronous Position)

Index	Name	Attribute	Type	Unit
6040	**Controlword**	Read-write	UINTEGER16	/
6060	**Modes of operation**	Read-write	INTEGER8	/
6067	**Position_window**	Read-write	UINTEGER32	Pulses (1 rev = 16,384 pulses)
6068	**Position_window_time**	Read-write	UINTEGER16	ms
6071	**Target_torque**	Read-write	INTEGER16	0.1% load ratio (1000 = 6 N·m)
607A	**Target_position**	Read-write	INTEGER32	Pulses (1 rev = 16,384 pulses)

Index	Name	Attribute	Type	Unit
6081	**Profile_velocity**	Read-write	UINTEGER32	0.1 rpm

Steps to Configure Position Mode (CSP):

1. While the motor is in the **disabled state (SWITCH_ON_DISABLED)**, set **Modes of operation** to **5**.
    - **Mandatory parameters**:
        - **Target_torque** (absolute max torque in position mode)
        - **Profile_velocity** (absolute speed in position mode)
    - **Optional parameters**:
        - **Position_window** (0 = disabled)
        - **Position_window_time** (0 = disabled)
2. Set **Controlword (6040)** to **15** to enable operation.
3. **Set Target_position** (absolute position) to move the motor to the desired position.

5.8. Velocity Mode

Index	Name	Attribute	Type	Unit
6040	**Controlword**	Read-write	UINTEGER16	/
6060	**Modes of operation**	Read-write	INTEGER8	/
6071	**Target_torque**	Read-write	INTEGER16	0.1% load ratio (1000 = 6 N·m)
60FF	**Target_velocity**	Read-write	INTEGER32	0.1 rpm

Steps to Configure Velocity Mode:

1. While the motor is in the **disabled state (SWITCH_ON_DISABLED)**, set **Modes of operation** to **3**.
    - **Mandatory parameter**:
        - **Target_torque** (absolute max torque in velocity mode)
3. Set **Controlword (6040)** to **15** to enable operation.

3. **Set Target_velocity** to reach the desired speed.

5.9. Torque Mode

Index	Name	Attribute	Type	Unit
6040	**Controlword**	Read-write	UINTEGER16	/
6060	**Modes of operation**	Read-write	INTEGER8	/
6071	**Target_torque**	Read-write	INTEGER16	0.1% load ratio (1000 = 6 N·m)

Steps to Configure Torque Mode:

1. While the motor is in the **disabled state (SWITCH_ON_DISABLED)**, set **Modes of operation** to **4**.
2. Set **Controlword (6040)** to **15** to enable operation.
3. **Set Target_torque** to output the desired torque.

5.10. Protocol Switching (Extended Frame): Switch Motor Protocol (Takes Effect After Power Cycle)

Data Field	29-bit ID	8-Byte Data Area
Size	Bit 28~0	Byte 0~6
Description	**0xFFF**	01 02 03 04 05 06 F_CMD

- **F_CMD** (Byte 6) defines the motor protocol:
  - **0**: Private protocol (default)
  - **1**: CANopen protocol
  - **2**: MIT protocol

Data Field	11-bit ID	8-Byte Data Area
Size	Bit 10~0	Byte 0~7
Description	Motor ID	64-bit MCU unique identifier

Response Frame:

6. MIT Communication Protocol Description

The motor communication adopts the CAN 2.0 interface with a default baud rate of 1 Mbps. The baud rate can be modified by switching to the private protocol. The standard frame format is as follows:

Data Field	11-bit ID		8-byte Data Area
Size	Bit 10~8	Bit 7~0	Byte 0~7
Description	Mode type	ID	

- Supported Control Modes:
- **MIT Mode**: Provides five motion control parameters to the motor.
- **Velocity Mode**: Specifies the target speed for the motor.
- **Position Mode**: Specifies the target position and speed, allowing the motor to run to the designated position at the configured speed.

6.1. Response Command 1: Data Feedback (Motor Status)

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	Byte 0~7
Description	Host ID	Byte 0: Motor CAN ID Byte 1~2: The current angle [0~65535], corresponds to (-12.57 rad ~ 12.57 rad) Byte 3 (high 8 bits),Byte 4[7-4] (low 4 bits): The current speed [0~4096], corresponds to (-44 rad/s ~ 44 rad/s) Byte 4[3-0] (high 4 bits),Byte 5 (low 8 bits): The current torque [0~4096], corresponds to (-17 N·m ~ 17 N·m) Byte 6~7: Winding temperature (in degrees)： Temp(Celsius) *10

6.2. Response Command 2: MCU Identification

Data Field	11-bit ID	8-byte Data Area

Size	Bit 10~0	Byte 0~7
Description	Motor ID	64-bit MCU unique identifier

6.3. Command 1: Enable Motor Operation

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	Byte 0~7
Description	Target motor CAN ID	FF FF FF FF FF FF FF FC

Response: Response Command 1

6.4. Command 2: Stop Motor Operation

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	Byte 0~7
Description	Target motor CAN ID	FF FF FF FF FF FF FF FD

Response: Response Command 1

6.5. Command 3: MIT Dynamic Parameters

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	Byte 0~1: Target angle [0~65535], (-12.57 rad ~ 12.57 rad)Byte 2 (high 8 bits), Byte 3[7-4] (low 4 bits): Target speed [0~4096], (-44 rad/s ~ 44 rad/s)Byte 3[3-0] (high 4 bits), Byte 4 (low 8 bits): Kp [0~4096], (0~500)Byte 5 (high 8 bits), Byte 6[7-4] (low 4 bits): Kd [0~4096], (0~5)Byte 6[3-0] (high 4 bits), Byte 7 (low 8 bits): Target torque [0~4096], (-17 N·m ~ 17 N·m)

Response: Response Command 1

6.6. Command 4: Set Zero Position (Non-Position Mode)

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	Byte 0~7

Description	Target motor CAN ID	FF FF FF FF FF FF FF FE

Response: Response Command 1

6.7. Command 5: Clear Errors & Read Fault Status

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	FF FF FF FF FF FF F_CMD FBF_CMD:- 0xFF → Clear current fault- Any other value → Returns fault value in Byte 1 of the response

Response (Fault Clear): Response Command 1

Fault Status Response:

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	Byte 0: Motor CAN IDByte 1~4: Fault value (Non-zero: Fault present; 0: Normal)Bit 14: Stall/I²t overload fault<Bit 7: Encoder not calibrated Bit 3: Overvoltage faultBit 2: Undervoltage faultBit 1: Driver IC faultBit 0: Motor overtemperature fault (Default threshold: 103°C)

6.8. Command 6: Set Operation Mode

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	FF FF FF FF FF FF F_CMD FCF_CMD: Mode type- 0: MIT mode (default)- 1: Position mode- 2: Velocity mode

Response: Response Command 1

6.9. Command 7: Modify Motor CAN ID

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	FF FF FF FF FF FF F_CMD FAF_CMD: Target motor CAN ID

Response: Response Command 2

6.10. Command 8: Change Communication Protocol (Takes Effect After Power Cycle)

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	FF FF FF FF FF FF F_CMD FDF_CMD: Protocol type- 0: Private protocol (default)- 1: CANopen - 2: MIT protocol

Response: Response Command 2

6.11. Command 9: Modify Host CAN ID

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	FF FF FF FF FF FF F_CMD 01F_CMD: Host CAN ID

Response: Response Command 2

6.12. Command 10: Position Mode Control Command

Data Field	11-bit ID		8-byte Data Area
Size	Bit 10~8	Bit 7~0	Byte 0~3: Target position (rad, 32-bit float)Byte 4~7: Target speed (rad/s, 32-bit float)
Description	1	Target motor CAN ID	

Response: Response Command 1

6.13. Command 11: Velocity Mode Control Command

Data Field	11-bit ID		8-byte Data Area
Size	Bit 10~8	Bit 7~0	Byte 0~3: Target speed (rad/s, 32-bit float) Byte 4~7: Current limit in speed/position mode (A, 32-bit float)
Description	2	Target motor CAN ID	

Response: Response Command 1

6.14. Command 12:Save motor data (requires firmware update to 0.1.3.14)

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	Byte0~Byte7
Description	Target motor CANID	FF FF FF FF FF FF FF F8

Response frame: Response command 2

6.15. Command 13: Motor actively reports (requires firmware update to 0.1.3.14)

Data Field	11-bit ID	8-byte Data Area
Size	Bit 10~0	Byte0~Byte7
Description	Target motor CANID	FF FF FF FF FF FF F_CMD F9 The F_CMD byte indicates the motor protocol type. 0 indicates no reporting (default). 1 is for reporting

Response frame: Response command 1

6.16. Command 14: Read motor parameters (requires firmware update to 0.1.3.14)

Data Field	11-bit ID		8Byte 数据区
Size	bit10~8	bit7~0	Byte0~Byte7
	3	7F	05 70 00 00 00 00 00 00
Description	3	Target motor CANID	Byte0~1: index(See the read/write parameter table below for details.) Byte2~3: 00 Byte4~7: 00 The above data is in low byte first, high byte last.

Response frame:

Data Field	11-bit ID		8Byte 数据区
Size	bit10~8	bit7~0	Byte0~Byte7
	3	7F	05 70 00 00 00 00 00 00
Description	3	Target motor CANID	Byte0~1: index See the read/write parameter table for details (same as the private protocol parameter table). Byte2~3: 00 Byte4~7: 00 For parameter data, data of 1 byte or more is formatted with the least significant byte first and the most significant byte last.

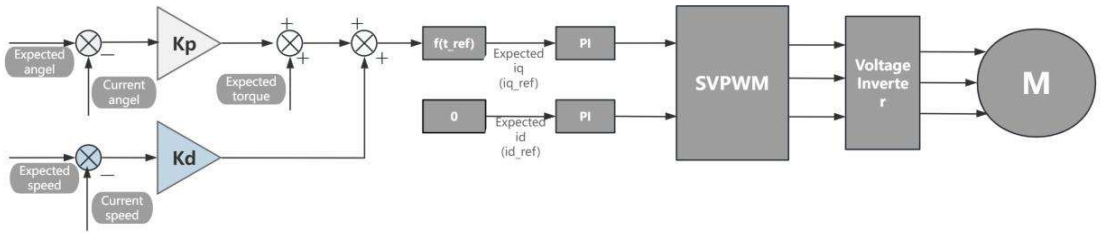
6.17. Command 15: Write motor parameters (requires firmware update to 0.1.3.14)

Data Field	11-bit ID		8Byte 数据区
Size	bit10~8	bit7~0	Byte0~Byte7
	4	7F	29 70 00 00 00 00 00 00
Description	4	Target motor CANID	Byte0~1: index See the read/write parameter table for details (same as the private protocol parameter table). Byte2~3: 00 Byte4~7: 00 Parameter data: The above data is in low byte first, high byte last.

Response frame:

Data Field	11-bit ID		8Byte 数据区
Size	bit10~8	bit7~0	Byte0~Byte7
	4	7F	29 70 00 00 00 00 00 00
Description	4	Target motor CANID	Byte0~1: index See the read/write parameter table for details (same as the private protocol parameter table). Byte2~3: 00 Byte4~7: 00 Parameter data: The above data is in low byte first, high byte last.

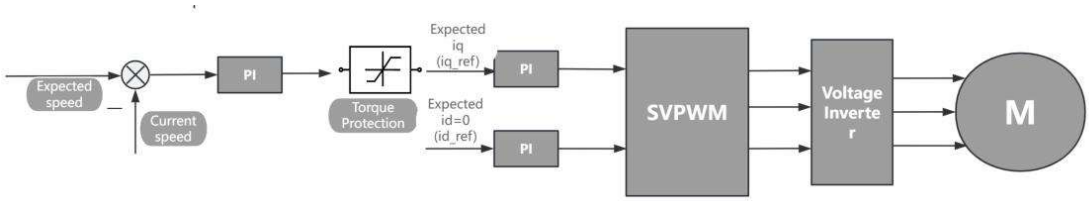
6.18. Motion Control Mode



The motor defaults to Motion Control Mode upon power-up.

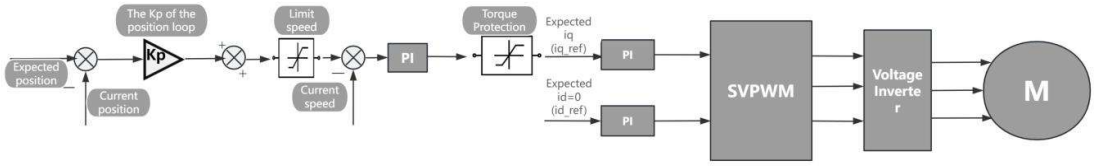
1. Send the *Motor Enable Command* (Command 1).
2. Send the *Motion Control Command* (Command 3) to activate dynamic parameter control.
3. Send the *Motor Stop Command* (Command 2) to halt operation when needed.

6.19. Velocity Mode



1. Configure the motor's operation mode by sending *Set Operation Mode Command* (Command 6) with **Mode = 2 (Velocity Mode)**.
2. Send the *Motor Enable Command* (Command 1) to activate the motor.
3. Send the *Velocity Mode Control Command* (Command 11) to set the **maximum current (absolute value)** and **target speed**.
4. To stop, send the *Motor Stop Command* (Command 2).

6.20. Position Mode (CSP - Cyclic Synchronous Position)



1. Configure the motor's operation mode by sending *Set Operation Mode Command* (Command 6) with **Mode = 1 (Position Mode)**.
2. Send the *Motor Enable Command* (Command 1) to activate the motor.
3. Send the *Position Mode Control Command* (Command 10) to set the **maximum speed (absolute value)** and **target position**.
4. To stop, send the *Motor Stop Command* (Command 2).

7. Version History

Version Number	Description	Date
1.0	Initial Release	November 25, 2025